

Quantum Computing and Static Analysis: State of the Art and Research Opportunities

Greta Dolcetti*

Ca' Foscari University of Venice
Venice, Italy
greta.dolcetti@unive.it

Giulio Zizzo

IBM Research Ireland
Dublin, Ireland
giulio.zizzo2@ibm.com

Vincenzo Arceri

University of Parma
Parma, Italy
vincenzo.arceri@unipr.it

Sergio Maffei

Imperial College London
London, United Kingdom
sergio.maffei@imperial.ac.uk

Agostino Cortesi

Ca' Foscari University of Venice
Venice, Italy
cortesi@unive.it

Abstract

Due to its fast evolution, quantum computing is increasingly becoming a software problem, one that requires particular care because of the intrinsic peculiarities of quantum programs. Static analysis for quantum programs is still in its infancy. It has produced both principled semantic techniques and practical pattern-based linters, but the two are not connected yet, resulting in limited adoption.

In this paper, we provide a concise presentation of the current state of the art, identify the challenges for static analysis of quantum programs, and propose a programmatic research agenda to address them, pairing each challenge with a concrete first step. Rather than surveying in depth existing analyses, this paper argues that the current generation of quantum static analysis tools is still missing the abstractions and infrastructures that enabled the success of static analysis for classical software. We identify these missing ingredients and outline a research roadmap for the next generation of quantum program analyzers.

CCS Concepts: • **Software and its engineering** → **Software verification**; **Automated static analysis**; • **Computer systems organization** → **Quantum computing**; • **Theory of computation** → **Program analysis**.

Keywords: Quantum Computing, Static Analysis, Abstract Interpretation, Abstract Domains, Code Smells, Linters

ACM Reference Format:

Greta Dolcetti, Giulio Zizzo, Vincenzo Arceri, Sergio Maffei, and Agostino Cortesi. 2026. Quantum Computing and Static Analysis: State of the Art and Research Opportunities. In *Proceedings of the 11th ACM*

*Work done while at IBM Research.



This work is licensed under a Creative Commons Attribution 4.0 International License.

NSAD '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2933-1/2026/10

<https://doi.org/10.1145/3840563.3843709>

SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains (NSAD '26), October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3840563.3843709>

1 Introduction

The evolution of quantum computing is finally starting to leave experimental laboratories and enter the real world, with a growing number of developers and diverse applications, which has led to the development of quantum programming languages and frameworks, such as Qiskit [28], Cirq [18], PennyLane [6], and Q# [46], as well as high-level languages like Guppy [32], Silq [8], and Qrisp [42]. Quantum computing companies generated more than \$1 billion in revenue worldwide in 2025, a figure expected to grow to as much as \$4.4 billion by 2028, with projections estimating that quantum computing could deliver between \$1.3 trillion and \$2.7 trillion in economic value to companies worldwide by 2035 [44].

It is evident that, given the importance that this sector will have, being capable of providing guarantees regarding quantum applications will be an important advantage in the market and will also improve the quality, safety, and trustworthiness of quantum applications. Nevertheless, the usage of static analysis for quantum computing is still largely underexplored and sparingly adopted: these two aspects, along with the potential impact of advancements in this area, represent the core motivation for this paper.

The aim of this paper is to: (i) provide a concise presentation of the current state of the art of quantum program analysis; (ii) distill the main challenges that currently limit its adoption; and (iii) propose motivated directions for the future in the form of a research agenda, where each item comes with a concrete first step.

Paper Structure. The paper is structured as follows. Section 2 presents the state of the art of static analysis for quantum programs, illustrating the two different existing approaches. Section 3 digs deeper into quantum-specific challenges for program analysis and their context. Section 4

represents the core of the paper and, for each challenge presented in the previous section, proposes an agenda item with actionable insights and proposed first steps. We discuss the limitations of the agenda and the paper in Section 5, and the paper is concluded by Section 6.

2 State of the Art

Quantum computing is evolving at a rapid pace and its analysis is becoming a software, hardware and infrastructure problem. The field is witnessing a growing developer base with diverse backgrounds (mathematics, physics, computer science, etc.), and a plethora of frameworks each with its own peculiarities. Among the most popular frameworks we can find: Qiskit [28], a mature and widely adopted open-source Python framework developed by IBM; PennyLane [6], a Python framework specifically tailored for quantum machine learning and quantum chemistry; and Cirq [18], a Python library developed by Google for writing, manipulating, and optimizing quantum circuits. While these frameworks are laying the groundwork for a new generation of quantum software, they are also introducing new, quantum-specific bugs [38] that are invisible to classical tools, and that can be extremely hard to detect and fix. A large-scale analysis of issues in quantum software repositories found that roughly a third of them are quantum-specific [47]. Some tools have been proposed, but their maturity and applicability are still limited, and they are not yet widely adopted by the quantum community.

2.1 What Makes Quantum Programs Different

Quantum programs are different from classical ones in several ways. First, quantum programs operate on qubits, which can exist in a superposition of states: the state of an n -qubit program is a unit vector $|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$ in the Hilbert space $\mathbb{C}^{2^n}$, described by exponentially many complex amplitudes. For example the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$ is a superposition of the two states $|00\rangle$ and $|11\rangle$, with equal amplitudes $1/\sqrt{2}$, and no amplitude for the other two states $|01\rangle$ and $|10\rangle$. Explicit state representations, exhaustive path enumeration, and even concrete collecting semantics are therefore intractable beyond toy examples. Second, the no-cloning theorem prevents unknown quantum states from being copied because no unitary U satisfies $U(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |\psi\rangle$ for every state $|\psi\rangle$, marking a fundamental difference from classical information: analyses must therefore treat quantum variables as consumable resources, tracking usage rather than value equality. Third, implicit measurement can occur when a qubit is discarded, which can lead to unintentional measurement and loss of information: dead code elimination, a usually safe optimization in classical programs, can therefore have a big impact in quantum programs. Finally, entanglement can create non-local correlations between qubits, which can make it difficult to reason about the behavior

of the program: an entangled state, such as the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$, cannot be factored as a tensor product $|\psi_1\rangle \otimes |\psi_2\rangle$ of the states of its qubits, so the state of an entangled group does not allow for a meaningful and safe decomposition. We refer the interested reader to [36] for a more in-depth discussion of these concepts.

In Listing 1, we show a small Qiskit program containing two silent issues: an operation performed after measurement and a gate applied after transpilation. Since neither issue raises an error or warning, they may be difficult to detect and fix, highlighting the need for effective static analysis tools that identify such problems in quantum programs before execution.

Listing 1. A small Qiskit program with two silent issues.

```
1 from qiskit import QuantumCircuit, transpile
2 from qiskit_aer import AerSimulator
3 qc = QuantumCircuit(2)
4 qc.h(0)
5 qc.cx(0, 1)
6 qc.measure_all()
7 qc.h(0) # Operation after measurement
8 backend = AerSimulator()
9 transpiled_qc = transpile(qc, backend)
10 transpiled_qc.h(1) # Gate applied after
    transpilation
```

2.2 Analyzing Quantum Programs

We notice that static analysis for quantum programs follows two main tracks: the semantic track and the practical track. A comprehensive technical survey of these approaches is provided in [1]; here we deliberately keep the presentation minimal and instead focus on the gap between principled techniques and developer practice, providing an adoption lens instead of a methodological one. In this paper we do not cover deductive verification approaches, such as quantum Hoare logic [33] and automated deductive verification frameworks like Qbricks [12] or SQIR/VOQC [24, 25], nor circuit equivalence checking tools like QCEC [10]. These approaches require user specifications or proofs, whereas our focus is on push-button static analysis that can be applied without extensive manual intervention. For a more comprehensive survey of these approaches, we refer the interested reader to [13].

2.2.1 The Semantic Track. In this track, two main approaches can be identified, as noted in [1]:

- *Flow-based analyses*, closer in spirit to classical program analysis, which employ dataflow analyses [4] and type checkers [8, 15, 26].
- *Domain-based analyses*, rooted in quantum information theory, which design abstract domains to reason about properties of quantum programs and computation, typically within the Abstract Interpretation framework [16]. Entanglement is the most studied

property [3, 5, 40], but abstractions of the whole quantum state during the computation have also been explored [9, 49].

Although these approaches provide strong formal guarantees, the resulting analyzers are typically prototypes evaluated on small benchmarks of textbook circuits and are not yet widely adopted by the community.

2.2.2 The Practical Track. As opposed to the semantic track, the practical track considers quantum software as a whole, and it often relaxes some guarantees in favor of a broader view. The proposed tools are usually bug-pattern linters that can be used by developers to catch common bugs and issues in their code, such as QChecker [50] (a tool performing AST-based checks over Qiskit programs), LintQ [39] (a set of ten analyses that detect likely bugs on Qiskit programs using CodeQL¹), and QSmell [14] (a tool for executing dynamic or static analyses to detect quantum-specific code smells). These tools often seek a compromise between soundness and usability, and they are designed to be easy to use and to integrate into the development workflow. Moreover, being tightly coupled to the frameworks they target, they are exposed to the frameworks' rapid evolution, which can make them hard to maintain and keep up-to-date.

2.2.3 The Gap to be Bridged. The gap between the two tracks above is evident: while the semantic track provides strong guarantees, it is often limited to small benchmarks and is not widely adopted by the community, while the practical track is more developer-friendly but lacks formal guarantees and can be affected by the rapid evolution of the frameworks. This gap highlights the need for a middle ground that combines the strengths of both tracks, providing lightweight, semantics-aware, developer-facing analysis tools that can be widely adopted by the community.

Moreover, many of the aspects that have contributed to the success of classical static analysis are still missing in the quantum domain, as will be discussed in the next paragraphs. For instance, shared intermediate representations (IRs) like LLVM have facilitated the development of a wide range of analysis tools in classical computing, while QIR is still marginally adopted for quantum analysis. Benchmark suites and competitions, such as SV-COMP [7] and Defects4J [30], have provided a standardized way to evaluate and compare classical analysis tools, whereas Bugs4Q [52] suffers from reproducibility issues and lacks a comprehensive benchmark suite. The culture of integrating linters into the development workflow, exemplified by tools like pylint² and ESLint³, has been instrumental in promoting best practices in classical software development, while quantum analysis tools remain largely standalone research prototypes. Finally,

industrial adoption stories, such as Astrée [17] at AIRBUS and Infer [11] at Meta, have demonstrated the practical value of classical static analysis in real-world settings, yet no similar success stories exist for quantum static analysis. These gaps highlight the need for a community effort to develop shared infrastructures and best practices that can support the growth of quantum static analysis.

3 Challenges

In this section, we discuss the main challenges that arise in the static analysis of quantum programs. These challenges stem from the unique characteristics of quantum computation, the fragmented and fast-moving landscape of quantum programming languages, and the limitations of existing analysis tools. Each challenge is presented with a brief explanation of its implications and a pointer to the corresponding agenda items that address it. For orthogonal quantum software engineering challenges, we refer the reader to [34].

C1: Exponential State Spaces. The nature of quantum computation (exponential state spaces, entanglement, and superposition) makes it challenging to design static analysis tools that can provide useful guarantees while remaining computationally feasible: a pure state of n qubits requires 2^n complex amplitudes, and a mixed state (density matrix) up to $4^n - 1$ real parameters, so any practical abstract domain must inevitably trade precision for tractability. The existing abstract domains often focus on specific properties of quantum programs, but they struggle to capture the full complexity of quantum states and operations. This limitation highlights the need for new abstract domains that can effectively reason about multiple properties simultaneously, while also being able to handle non-Clifford gates and maintain precision throughout the analysis.

C2: A Fragmented and Fast-Moving Language Landscape. Most of the analysis and tools are tied to specific quantum programming languages or frameworks, which can limit their applicability and adoption. The rapid evolution of these languages and frameworks, along with the lack of consensus on the appropriate level of abstraction for quantum programming, further complicates the development of static analysis tools. As a concrete example of the challenges posted by the fast-moving landscape, Qiskit 1.0 (released in February 2024) removed, renamed or deprecated significant parts of its public API, and similarly did Qiskit 2.0 when it was released in April 2025, about a year later. This challenge highlights the need for language-agnostic analysis techniques that can adapt to different quantum programming paradigms and remain effective in the face of ongoing changes in the quantum software landscape. The Quantum Intermediate Representation (QIR)⁴ is a promising step in this direction, as it provides a common representation for

¹<https://codeql.github.com/>

²<https://github.com/pylint-dev/pylint>

³<https://eslint.org/>

⁴<https://github.com/qir-alliance/qir-spec>

quantum programs that can facilitate the development of language-agnostic analysis tools. Similarly [31] proposes a method to analyze quantum and classical computing code in a unified representation. However, their usage is still limited for analysis purposes, and further work is needed to fully leverage their potential in this context.

C3: New Paradigms. While quantum computation has been primarily focused on traditional quantum algorithms, emerging paradigms such as quantum machine learning might introduce new interesting abstractions and properties that are not yet supported by existing static analysis tools. These new paradigms require the development of novel abstract domains and analysis techniques that can effectively reason about the unique characteristics of quantum machine learning models, such as parameterized circuits and hybrid classical-quantum loops, as well as properties like noise resilience or quantum adversarial robustness. Empirical evidence confirms that quantum machine learning frameworks come with their own classes of defects [51], while early work on the formal verification of variational quantum circuits [2] addresses only a fraction of the properties of interest.

C4: A Broken Testing Loop. The probabilistic nature of quantum computation, along with the destructive nature of quantum measurements, and the impact of noise make it difficult to validate the correctness of quantum programs through traditional testing methods. Runtime approaches such as statistical assertions [27] mitigate the problem only partially, as they still rely on repeated executions. The issue here lies in the concrete cost of testing: the number of shots required to distinguish the observed output distribution from the expected one might be prohibitively high, especially for programs with a large number of qubits or complex entanglement structures. The limitations of the testing loop increase the importance of static analysis tools that can provide sound guarantees about program behavior. However, it also presents a challenge in validating the analyses themselves, as there is no straightforward way to confirm their correctness through empirical testing.

C5: The Semantics Gap: Noise and Transpilation. The behavior of quantum programs can be affected by various factors, such as noise in the hardware and transformations applied during transpilation [45]. These factors can introduce discrepancies between the intended semantics of a program and its actual execution, which can lead to unexpected bugs and issues depending on which version has been executed and, similarly, analyzed. This challenge emphasizes the need for static analysis tools that can account for these discrepancies and provide sound guarantees about program behavior, even in the presence of noise and transpilation effects.

C6: LLM-Generated Quantum Code. Although Large Language Models (LLMs) have shown promise in generating code for various programming languages, their performance

in generating quantum code is still limited [48]. The scarcity of training data, the rapid evolution of quantum programming languages and APIs, and the unintuitive semantics of quantum computation contribute to the challenges faced by LLMs in this domain. As a result, LLM-generated quantum code may contain subtle bugs that are difficult to detect using traditional testing methods, further highlighting the need for robust static analysis tools that can help developers identify and address these issues and prove the correctness of the generated code.

C7: No Common Ground for Evaluation. Whereas for classical software there exist large and well-curated datasets of bugs and benchmarks for evaluating static analysis tools, the quantum software community lacks such resources. The limited availability of bug datasets such as [52], which nonetheless suffers important reproducibility issues [37], and the absence of standardized evaluation frameworks make it difficult to assess the effectiveness of static analysis tools and compare their performance. This challenge underscores the need for the development of common evaluation benchmarks and property exchange formats that can facilitate the assessment and comparison of different static analysis approaches in the quantum domain.

C8: Heterogeneous Developer Backgrounds. Similarly to the background of many developers in the data science and machine learning communities, quantum software developers often come from diverse backgrounds, including physics, chemistry, and mathematics, with varying levels of experience in software engineering. This heterogeneity is one of the reasons of the success of quantum computing on one hand but the possible cause of mistakes and incomprehensions on the other. Likewise, quantum software development often relies on notebook interactive development environments (e.g., Jupyter) because of the ease of the prototyping phase, but introduces challenges as the possibility of running instructions in a non-sequential order.

4 Research Opportunities

In the previous section, we outlined the main challenges that arise in the static analysis of quantum programs. In this section, we present a research agenda that addresses these challenges and outlines potential directions for future work. Each agenda item is designed to tackle specific challenges and proposes a first doable step to advance the state of the art in quantum program analysis.

A1: The Missing Abstract Domains [C1, C3, C5]. Not many abstract domains have been proposed for quantum programs, and those that exist often focus on specific properties, such as entanglement [3, 5, 40]. To address this gap, we propose the development of new abstract domains that can capture a wider range of properties and relationships in quantum programs. This includes relational domains that

connect classical and quantum variables in hybrid programs, phase/amplitude information beyond Clifford gates, reduced products of entanglement and value domains, widening operators for measurement-guarded loops, and noise-parametric domains where transformers carry an error budget. Moreover, new specific domains related to quantum machine learning will be needed to reason about specific properties of those models. A first step in this direction would be to develop a reduced product $D_E \otimes D_N$ of an existing entanglement domain D_E with a classical numerical domain D_N over the shared measurement variables, and evaluate it on Variational Quantum Algorithm (VQA)-style loops.

A2: A High-Level Semantic Linter for Quantum Programming [C4, C6, C8]. As shown in Section 2, there exists a gap between the semantic track and the practical one. To bridge this space, we suggest exploring a high-level linter for the analysis of quantum programs grounded in formal guarantees to be used when testing is not enough. Similarly to what was done in other fields, for example PYRA [20–22] and flowR [43] for data science, the idea is to abstractly reason on quantum programs (for example by inferring abstract quantum datatypes), running checkers on top of the analysis and raising warnings along with their cause and possible mitigation. This would differ in nature from most of the approaches presented in the practical track because it would be based on abstract domains rather than pattern recognition. A first step for the future would be creating a validated catalog of quantum code smells beyond [14], assembled by mining public quantum projects, studying best practices and interacting with quantum scientists.

A3: New Paradigms and Generalizability [C2, C3]. As detailed before, quantum programs are often hybrid in nature, especially with new chemical and machine learning applications; nevertheless, they are usually targeted only in their quantum nature. To mitigate this aspect, we propose treating them as a whole: propagating and computing both quantum and classical information. Moreover, the language-analyzer coupling is now limiting and generalizable approaches such as QIR are not yet widely adopted, therefore we suggest an approach similar to LiSA [35], a generic framework where modular multilanguage static analyses can be defined through the abstract interpretation theory, using different frontends to obtain a uniform representation and language-specific semantics, while exploiting the common features of the analysis. A first step in this direction would be integrating a QIR frontend in LiSA and using it to analyze a simple hybrid program, such as a Variational Quantum Algorithm (VQA), with a classical domain and an entanglement domain. We prioritize QIR because, being LLVM-based, it represents classical control flow natively, meaning that hybrid programs require not too much work, and because it is designed as a cross-vendor target.

A4: Analyses Placed Along the Compilation Pipeline [C5]. Some properties of quantum programs are meaningful only at specific stages of the compilation pipeline. Before transpilation, analyses can capture high-level intent (e.g., algorithmic structure and code smells), while after transpilation the same program exposes low-level properties (e.g., gate counts, depth, routing overhead, and hardware-aware entanglement) that are invisible earlier. This motivates analysis placement: for each property, we should identify (i) the earliest stage where checking is sound and (ii) the stage where information is most precise and actionable. A concrete first step is to pick one property (for instance, an entanglement abstraction), one widely used transpilation pass (e.g., basis-gate rewriting or qubit routing), and then characterize three outcomes: what is preserved exactly, what is safely weakened, and what must be recomputed.

A5: Static Analysis in the LLM Loop [C6]. As discussed in the previous section, LLMs are still far from being capable of generating correct complete quantum programs. Nevertheless, many papers such as [19, 29] have shown how the combination of static analyzers and LLMs could improve code generation. This research item is bidirectional: first, the analyzer’s feedback could be used in the generate-check-repair loop to improve LLM-based quantum program generation, both in inference and training; conversely, an LLM rooted in quantum information, for example when coupled with a RAG, could be used to propose candidate smells and fixes, possibly coupled with a semantic-based linter. A first step in this direction would be running existing analyzers on the few available benchmarks to assess how many programs deemed correct by tests still violate semantic checks. Concretely, at inference time analyzer warnings, together with their cause and suggested mitigation, are fed back as structured context within a generate-check-repair loop, while at training time analyzer verdicts act as an additional reward or filtering signal on top of executable tests.

A6: Shared Corpora, Properties, and a Friendly Competition [C4, C7, C8]. One of the strengths of popular languages such as Python is the valuable community material. In quantum computing, a shared curated corpus of real-world analyzable quantum programs annotated with known issues along with a sharable notation format is still missing. To fill this gap, we call for a shared community effort in consolidating and validating existing benchmarks and, more importantly, in creating new shared benchmarks and positive competition in a similar manner to SV-COMP [7]. A first step in this direction would be the definition of a minimal property exchange format, e.g. a JSON or YAML file recording the logical-to-physical qubit mapping, the entanglement partition, and the probability a tool claims at a given pipeline stage.

Table 1. Summary of the proposed research agenda for quantum program analysis: each item lists its description and the first actionable step.

Item	Description	First Step
A1	Missing abstract domains	Combine an entanglement domain with a classical numerical domain over shared measurement variables; evaluate on VQA-style loops.
A2	High-level semantic linter for quantum programming	Build a validated catalog of quantum code smells beyond [14] by mining public projects, best practices, and feedback from quantum scientists.
A3	New paradigms and generalizability	Integrate a QIR frontend in LiSA and analyze a simple hybrid program with classical and entanglement domains.
A4	Analyses along the compilation pipeline	Track one property across a standard transpilation pass and characterize what survives.
A5	Static analysis in the LLM loop	Run existing analyzers on available benchmarks to measure how many test-passing programs still violate semantic checks and adopt a repair loop.
A6	Shared corpora and competition	Consolidate and validate benchmarks, then create shared benchmarks and friendly competition in the style of SV-COMP [7].

5 Constraints and Limitations

Due to the intrinsic nature of quantum computation, static analysis of quantum programs faces several constraints and limitations. First, as in the classical setting, no static analysis can be both sound and complete for non-trivial semantic properties; quantum programs inherit this limitation from their classical host languages. Second, the precision/cost trade-offs are harsher than in classical static analysis. Third, the state of quantum computing, now still largely in the Noisy Intermediate-Scale Quantum (NISQ) [41] stage, makes it almost impossible to reason only statically about quantum programs, as noise and hardware drift ultimately escape purely static reasoning. Finally, turning to the present paper, each research agenda item is presented independently, although we highlight their interconnections, so progress might be uneven. Moreover, this agenda is deliberately programmatic: no prototype or feasibility study accompanies it, and the technical proposals are sketched rather than developed. In addition, in this paper we focus on static analysis for quantum programs, while the usage of quantum computing for static analysis is left out of scope. Nevertheless, we refer the interested reader to preliminary work in this direction [23].

6 Conclusions

In this paper, we provided an overview of the existing state of the art for quantum program analysis: two different approaches exist, but there is a gap between them and they do not entirely address all the peculiarities of quantum computing. To address this issue, we propose the research agenda presented in Section 4, which is designed to tackle the challenges identified in Section 3 and to advance the state of the art in quantum program analysis. For each item, we provide a brief description and a concrete first step that can be taken to make progress in that area, as summarized in Table 1.

The idea is not only to provide a roadmap for future research in quantum program analysis, but to make it actionable and concrete.

Acknowledgments

Work partially supported by PR Veneto FESR 2021-2027, Az. 1.4.1 ID 251382_008101 "SmartCycle – Cybersecurity" (CUP: H73C25001340002), by Regione Veneto: RIR project SUPREME (CUP D19J24000810007) and by EU Horizon Europe: SecQDevOps Grant Agreement No. 101225776.

References

- [1] Nicola Assolini, Alessandra Di Pierro, and Isabella Mastroeni. 2026. Challenges in Quantum Programs Analysis. *International Journal on Software Tools for Technology Transfer* 28, 3 (2026), 345–374. doi:10.1007/s10009-026-00845-1
- [2] Nicola Assolini, Luca Marzari, Isabella Mastroeni, and Alessandra Di Pierro. 2025. Formal Verification of Variational Quantum Circuits. *CoRR* abs/2507.10635 (2025). doi:10.48550/ARXIV.2507.10635
- [3] Nicola Assolini, Alessandra Di Pierro, and Isabella Mastroeni. 2024. Abstracting Entanglement. In *Proceedings of the 10th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains, NSAD 2024, Pasadena, CA, USA, 22 October 2024*, Vincenzo Arceri and Michele Pasqua (Eds.). ACM, 34–41. doi:10.1145/3689609.3689998
- [4] Nicola Assolini, Alessandra Di Pierro, and Isabella Mastroeni. 2024. Static Analysis of Quantum Programs. In *Static Analysis - 31st International Symposium, SAS 2024, Pasadena, CA, USA, October 20–22, 2024, Proceedings (Lecture Notes in Computer Science, Vol. 14995)*, Roberto Giacobazzi and Alessandra Gorla (Eds.). Springer, 1–25. doi:10.1007/978-3-031-74776-2_1
- [5] Nicola Assolini, Alessandra Di Pierro, and Isabella Mastroeni. 2025. A Static Analysis of Entanglement. In *Verification, Model Checking, and Abstract Interpretation - 26th International Conference, VMCAI 2025, Denver, CO, USA, January 20–21, 2025, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 15530)*, Shankaranarayanan Krishna, Sriram Sankaranarayanan, and Ashutosh Trivedi (Eds.). Springer, 50–71. doi:10.1007/978-3-031-82703-7_3
- [6] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M Sohaib Alam, Guillermo Alonso-Linaje, Bharath AkashNarayanan, Ali Asadi, et al. 2018. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968* (2018).
- [7] Dirk Beyer and Jan Strejcek. 2026. Evaluating Software Verifiers for C, Java, and SV-LIB - (Report on SV-COMP 2026). In *Tools and Algorithms for the Construction and Analysis of Systems - 32nd International Conference, TACAS 2026, Turin, Italy, April 11–16, 2026, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 16506)*, Sebastian Junges and Guy Katz (Eds.). Springer, 461–502. doi:10.1007/978-3-032-22749-2_23
- [8] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: a high-level quantum language with safe uncomputation

- and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (*PLDI 2020*). Association for Computing Machinery, New York, NY, USA, 286–300. doi:10.1145/3385412.3386007
- [9] Benjamin Bichsel, Anouk Paradis, Maximilian Baader, and Martin T. Vechev. 2023. Abstraqt: Analysis of Quantum Circuits via Abstract Stabilizer Simulation. *Quantum* 7 (2023), 1185. doi:10.22331/Q-2023-11-20-1185
- [10] Lukas Burgholzer and Robert Wille. 2021. QCEC: A JKQ tool for quantum circuit equivalence checking. *Software Impacts* (Feb. 2021). doi:10.1016/j.simpa.2020.100051
- [11] Cristiano Calcagno and Dino Distefano. 2011. Infer: An Automatic Program Verifier for Memory Safety of C Programs. In *NASA Formal Methods - Third International Symposium, NFM 2011, Pasadena, CA, USA, April 18-20, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6617)*, Mihaela Gheorghiu Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi (Eds.). Springer, 459–465. doi:10.1007/978-3-642-20398-5_33
- [12] Christophe Charetton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An Automated Deductive Verification Framework for Circuit-building Quantum Programs. In *Programming Languages and Systems - 30th European Symposium on Programming, ESOP 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings (Lecture Notes in Computer Science, Vol. 12648)*, Nobuko Yoshida (Ed.). Springer, 148–177. doi:10.1007/978-3-030-72019-3_6
- [13] Christophe Charetton, Dongho Lee, Benoît Valiron, Renaud Vilmart, Sébastien Bardin, and Zhaowei Xu. 2023. Formal Methods for Quantum Algorithms. In *Handbook of Formal Analysis and Verification in Cryptography*, Sedat Akleylek and Besik Dundua (Eds.). CRC Press, 319–422. doi:10.1201/9781003090052-7
- [14] Qihong Chen, Rúben Câmara, José Campos, André Souto, and Iftekhar Ahmed. 2023. The Smelly Eight: An Empirical Study on the Prevalence of Code Smells in Quantum Computing. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 358–370. doi:10.1109/ICSE48619.2023.00041
- [15] Andrea Colledan and Ugo Dal Lago. 2025. Flexible Type-Based Resource Estimation in Quantum Circuit Description Languages. *Proc. ACM Program. Lang.* 9, POPL (2025), 1386–1416. doi:10.1145/3704883
- [16] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages, Los Angeles, California, USA, January 1977*, Robert M. Graham, Michael A. Harrison, and Ravi Sethi (Eds.). ACM, 238–252. doi:10.1145/512950.512973
- [17] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTREÉ Analyzer. In *Programming Languages and Systems, 14th European Symposium on Programming, ESOP 2005, Edinburgh, UK, April 4-8, 2005, Proceedings (Lecture Notes in Computer Science, Vol. 3444)*, Shmuel Sagiv (Ed.). Springer, 21–30. doi:10.1007/978-3-540-31987-0_3
- [18] Cirq Developers. 2026. *Cirq*. doi:10.5281/zenodo.21080138
- [19] Greta Dolcetti, Vincenzo Arceri, Eleonora Iotti, Sergio Maffei, Agostino Cortesi, and Enea Zaffanella. 2026. Helping LLMs improve code generation using feedback from testing and static analysis. *Discov. Artif. Intell.* 6, 1 (2026), 314. doi:10.1007/S44163-026-01009-5
- [20] Greta Dolcetti, Vincenzo Arceri, Antonella Mensi, Enea Zaffanella, Caterina Urban, and Agostino Cortesi. 2025. Introducing Pyra: A High-Level Linter for Data Science Software. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track and Demo Track - European Conference, ECML PKDD 2025, Porto, Portugal, September 15-19, 2025, Proceedings, Part X (Lecture Notes in Computer Science, Vol. 16022)*, Inês Dutra, Mykola Pechenizkiy, Paulo Cortez, Sepideh Pashami, Arian Pasquali, Nuno Moniz, Alípio M. Jorge, Carlos Soares, Pedro H. Abreu, and João Gama (Eds.). Springer, 449–453. doi:10.1007/978-3-032-06129-4_29
- [21] Greta Dolcetti, Vincenzo Arceri, Antonella Mensi, Enea Zaffanella, Caterina Urban, and Agostino Cortesi. 2026. PYRA : A high-level linter for data science software. *Knowl. Based Syst.* 337 (2026), 115412. doi:10.1016/J.KNOSYS.2026.115412
- [22] Greta Dolcetti, Agostino Cortesi, Caterina Urban, and Enea Zaffanella. 2024. Towards a High Level Linter for Data Science. In *Proceedings of the 10th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains, NSAD 2024, Pasadena, CA, USA, 22 October 2024*, Vincenzo Arceri and Michele Pasqua (Eds.). ACM, 18–25. doi:10.1145/3689609.3689996
- [23] Yicheng Guang, Pietro Zanotta, Kai Zhou, Yueqi Chen, and Ramin Ayanzadeh. 2025. On the Potential of Quantum Computing in Classical Program Analysis. In *IEEE International Conference on Quantum Computing and Engineering, QCE 2025, Albuquerque, NM, USA, August 30 - Sept. 5, 2025*, Marek Osinski, Brian La Cour, and Samanvay Sharma (Eds.). IEEE, 2364–2374. doi:10.1109/QCE65121.2025.00257
- [24] Kesha Hietala, Robert Rand, Shih-Han Hung, Liyi Li, and Michael Hicks. 2021. Proving Quantum Programs Correct. In *12th International Conference on Interactive Theorem Proving (ITP 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 193)*, Liron Cohen and Cezary Kaliszyk (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Article 21, 21:1–21:19 pages. doi:10.4230/LIPIcs.ITP.2021.21
- [25] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A Verified Optimizer for Quantum Circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (Jan. 2021), 29 pages. arXiv:1912.02250 doi:10.1145/3434318
- [26] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proc. ACM Program. Lang.* 9, POPL, Article 6 (Jan. 2025), 28 pages. doi:10.1145/3704842
- [27] Yipeng Huang and Margaret Martonosi. 2019. Statistical assertions for validating patterns and finding bugs in quantum programs. In *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*, Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman (Eds.). ACM, 541–553. doi:10.1145/3307650.3322213
- [28] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. *CoRR* abs/2405.08810 (2024). arXiv:2405.08810 doi:10.48550/ARXIV.2405.08810
- [29] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 1646–1656. doi:10.1145/3611643.3613892
- [30] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *International Symposium on Software Testing and Analysis, ISSTA '14, San Jose, CA, USA - July 21 - 26, 2014*, Corina S. Pasareanu and Darko Marinov (Eds.). ACM, 437–440. doi:10.1145/2610384.2628055
- [31] Maximilian Kaul, Alexander Küchler, and Christian Banse. 2023. A Uniform Representation of Classical and Quantum Source Code for Static Code Analysis. In *IEEE International Conference on Quantum Computing and Engineering, QCE 2023, Bellevue, WA, USA, September 17-22, 2023*, Brian La Cour, Lia Yeh, and Marek Osinski (Eds.). IEEE, 1013–1019. doi:10.1109/QCE57702.2023.00115
- [32] Mark Koch, Alan Lawrence, Kartik Singhal, Seyon Sivarajah, and Ross Duncan. 2025. GUPPY: Pythonic Quantum-Classical Programming. *CoRR* abs/2510.12582 (2025). doi:10.48550/ARXIV.2510.12582

- [33] Junyi Liu, Bohua Zhan, Shuling Wang, Shenggang Ying, Tao Liu, Yangjia Li, Mingsheng Ying, and Naijun Zhan. 2019. Formal Verification of Quantum Algorithms Using Quantum Hoare Logic. In *Computer Aided Verification - 31st International Conference, CAV 2019, New York City, NY, USA, July 15-18, 2019, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 11562)*, Isil Dillig and Serdar Tasiran (Eds.). Springer, 187–207. doi:10.1007/978-3-030-25543-5_12
- [34] Juan Manuel Murillo, José García-Alonso, Enrique Moguel, Johanna Barzen, Frank Leymann, Shaikat Ali, Tao Yue, Paolo Arcaini, Ricardo Pérez-Castillo, Ignacio García Rodríguez de Guzmán, Mario Piattini, Antonio Ruiz-Cortés, Antonio Brogi, Jianjun Zhao, Andriy V. Miranskyy, and Manuel Wimmer. 2025. Quantum Software Engineering: Roadmap and Challenges Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 154:1–154:48. doi:10.1145/3712002
- [35] Luca Negrini, Pietro Ferrara, Vincenzo Arceri, and Agostino Cortesi. 2023. LiSA: A Generic Framework for Multilanguage Static Analysis. In *Challenges of Software Verification*, Vincenzo Arceri, Agostino Cortesi, Pietro Ferrara, and Martina Oliaro (Eds.). Intelligent Systems Reference Library, Vol. 238. Springer, 19–42. doi:10.1007/978-981-19-9601-6_2
- [36] M.A. Nielsen and I.L. Chuang. 2010. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press. <https://books.google.it/books?id=s4DEy7o-a0C>
- [37] Haruto Ohto, Yuta Ishimoto, Shinsuke Matsumoto, and Shinji Kusumoto. 2026. On the Reproducibility of Quantum Software Defect Datasets: A Case Study of Bugs4Q. *CoRR* abs/2606.27124 (2026). arXiv:2606.27124 doi:10.48550/ARXIV.2606.27124
- [38] Matteo Paltenghi and Michael Pradel. 2022. Bugs in Quantum computing platforms: an empirical study. *Proc. ACM Program. Lang.* 6, OOPSLA1 (2022), 1–27. doi:10.1145/3527330
- [39] Matteo Paltenghi and Michael Pradel. 2024. Analyzing Quantum Programs with LintQ: A Static Analysis Framework for Qiskit. *Proc. ACM Softw. Eng.* 1, FSE (2024), 2144–2166. doi:10.1145/3660802
- [40] Simon Perdrix. 2008. Quantum Entanglement Analysis Based on Abstract Interpretation. In *Static Analysis, 15th International Symposium, SAS 2008, Valencia, Spain, July 16-18, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 5079)*, María Alpuente and Germán Vidal (Eds.). Springer, 270–282. doi:10.1007/978-3-540-69166-2_18
- [41] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (2018), 79. doi:10.22331/Q-2018-08-06-79
- [42] Raphael Seidel, Sebastian Bock, René Zander, Matic Petric, Niklas Steinmann, Nikolay Tcholtchev, and Manfred Hauswirth. 2024. Qrisp: A Framework for Compilable High-Level Programming of Gate-Based Quantum Computers. *CoRR* abs/2406.14792 (2024). doi:10.48550/ARXIV.2406.14792
- [43] Florian Sihler and Matthias Tichy. 2025. Statically Analyzing the Dataflow of R Programs. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 1034–1062. doi:10.1145/3763087
- [44] Henning Soller, Duc Nam Nguyen, Martina Gschwendtner, Victor Kermans, Waldemar Svejstrup, and Waris Ziarkash. 2026. *McKinsey Quantum Technology Monitor 2026: A Commercial Tipping Point*. Report. McKinsey & Company. <https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/mckinsey-quantum-technology-monitor-2026-a-commercial-tipping-point> Published April 28, 2026. Accessed: 2026-07-06.
- [45] Manuel De Stefano, Dario Di Nucci, Fabio Palomba, and Andrea De Lucia. 2024. An empirical study into the effects of transpilation on quantum circuit smells. *Empir. Softw. Eng.* 29, 3 (2024), 61. doi:10.1007/S10664-024-10461-9
- [46] Krysta M. Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher E. Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop, RWDSL@CGO 2018, Vienna, Austria, February 24-24, 2018*. ACM, 7:1–7:10. doi:10.1145/3183895.3183901
- [47] Krishna Upadhyay, Vinaik Chhetri, A. B. Siddique, and Umar Farooq. 2025. Analyzing the Evolution and Maintenance of Quantum Software Repositories. In *IEEE International Conference on Quantum Software, QSW 2025, Helsinki, Finland, July 7-12, 2025*, Rong N. Chang, Carl K. Chang, Jingwei Yang, Nimanthi Atukorala, Dan Chen, Sumi Helal, Sasu Tarkoma, Qiang He, Teflik Kosar, Claudio A. Ardagna, Sebastian Feld, Elisabetta Di Nitto, and Manuel Wimmer (Eds.). IEEE, 173–184. doi:10.1109/QSW67625.2025.00029
- [48] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit HumanEval: An Evaluation Benchmark for Quantum Code Generative Models. In *IEEE International Conference on Quantum Computing and Engineering, QCE 2024, Montreal, QC, Canada, September 15-20, 2024*, Marek Osinski, Brian La Cour, and Lia Yeh (Eds.). IEEE, 1169–1176. doi:10.1109/QCE60285.2024.00137
- [49] Nengkun Yu and Jens Palsberg. 2021. Quantum abstract interpretation. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 542–558. doi:10.1145/3453483.3454061
- [50] Pengzhan Zhao, Xiongfei Wu, Zhuo Li, and Jianjun Zhao. 2023. QChecker: Detecting Bugs in Quantum Programs via Static Analysis. In *4th IEEE/ACM International Workshop on Quantum Software Engineering, Q-SE@ICSE 2023, Melbourne, Australia, May 17, 2023*. IEEE, 50–57. doi:10.1109/Q-SE59154.2023.00014
- [51] Pengzhan Zhao, Xiongfei Wu, Junjie Luo, Zhuo Li, and Jianjun Zhao. 2023. An Empirical Study of Bugs in Quantum Machine Learning Frameworks. In *IEEE International Conference on Quantum Software, QSW 2023, Chicago, IL, USA, July 2-8, 2023*, Shaikat Ali, Claudio Agostino Ardagna, Nimanthi L. Atukorala, Johanna Barzen, Carl K. Chang, Rong N. Chang, Jing Fan, Ismael Faro, Sebastian Feld, Geoffrey Fox, Zhi Jin, Frank Leymann, Florian Neukart, Salvador de la Puente, and Manuel Wimmer (Eds.). IEEE, 68–75. doi:10.1109/QSW59989.2023.00018
- [52] Pengzhan Zhao, Jianjun Zhao, Zhongtao Miao, and Shuhan Lan. 2021. Bugs4Q: A Benchmark of Real Bugs for Quantum Programs. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 1373–1376. doi:10.1109/ASE51524.2021.9678908

Received 2026-07-14; accepted 2026-08-17